

Módulo 02 — Além do chat: System One e o Jev

Cada slide da apresentação explicado em linguagem simples: termos, o que falar e o que responder.

1. Slide a slide

Slide a slide

Este guia explica cada slide da apresentação em linguagem simples: o que significa cada termo, o que falar e o que responder se perguntarem. Leia uma vez inteiro antes da aula; durante a aula, as notas do PPTX bastam.

Se só der tempo de entender uma coisa: em muitos sistemas, a IA não precisa escrever um texto, só precisa **escolher** (sim/não, uma categoria, uma nota). Quando ela só escolhe, o resultado é mais fácil de usar no código. E se ela disser "tenho 99% de certeza", você precisa **conferir** se ela acerta 99% das vezes antes de acreditar.

Antes da aula: rode `pnpm -F @mentoria/curador m02:comparar:solucao` (leva uns 2 minutos) para a demo de calibração ter dados.

Slide 1 — Capa

Abertura. Diga: *"Na aula passada o modelo escrevia. Hoje ele vai decidir."*

Slide 2 — Hoje

Os quatro objetivos. Não leia a lista.

- **Decisão tipada:** uma resposta com formato fixo e conhecido, como `{"relevante": true}`, em vez de um parágrafo. "Tipada" vem de **tipo**, no sentido de programação: texto, número, verdadeiro/falso, uma opção de uma lista.
 - **Sinal de confiança:** um número (de 0 a 1) que diz o quanto o modelo tem certeza.
-

Slide 3 — O que vocês trouxeram da M1

Na tarefa da semana passada, o modelo respondia em **texto livre**. Com temperatura alta, ele mudou o jeito de escrever: às vezes "Relevante.", às vezes "Relevantes →", às vezes "JUSTIFICATIVA:" em maiúsculas.

O que falar: pergunte quem tentou fazer um programa ler essas respostas. O ponto é: se o formato muda, o programa que lê a resposta quebra ou, pior, lê errado sem avisar.

Slide 4 — A frase da aula

Muita "IA" em software não precisa gerar texto: precisa tomar uma decisão que o código consiga usar. Uma decisão útil tem tipo e confiança. E confiança só vale se for medida.

- **Tipo:** o programa sabe exatamente o formato da resposta (não precisa adivinhar).
- **Confiança:** o programa sabe quando a resposta é duvidosa e pode chamar uma pessoa.
- **Medida:** se o modelo diz 90%, ele precisa acertar mais ou menos 90% das vezes. Senão o número é enfeite.

Escreva a frase no chat.

Slide 5 — Divisória "1 · System One e o Jev"

Passe direto.

Slide 6 — Sistema 1 e Sistema 2

Referência ao livro *Rápido e Devagar*, do psicólogo Daniel Kahneman (prêmio Nobel de Economia).

- **Sistema 1:** o pensamento rápido e automático. Você reconhece o rosto de um amigo sem esforço.
- **Sistema 2:** o pensamento lento e cuidadoso. Fazer 17 × 24 de cabeça.

A ideia da TypeSafe: muitas decisões dentro de um software são do tipo "Sistema 1": olhar um texto e julgar rápido (é spam? é urgente?). Não precisam de um modelo que escreva um texto para chegar à resposta.

Cuidado (diga isso): no livro, o Sistema 1 é o que mais erra por vieses. A empresa afirma que o modelo deles é rápido **e** confiável. É uma afirmação deles; a aula vai mostrar como testar.

Slide 7 — O Jev, segundo o anúncio

O que a empresa TypeSafe AI diz sobre o produto. Tudo neste slide é **afirmação do fornecedor**.

- **Early access:** o produto está liberado só para alguns usuários (lista de espera). Não é um produto maduro.

- **Estado:** o conteúdo que o modelo analisa (um texto, uma conversa, um registro).
 - **Perguntas tipadas:** perguntas com formato de resposta definido ("escolha uma destas opções", "sim ou não").
 - **Em paralelo:** o LLM da Aula 1 escreve um pedaço de cada vez; o Jev, segundo eles, calcula todas as respostas de uma vez. Por isso seria mais rápido.
 - **RLCD:** o nome que eles deram ao método de treino ("aprendizado por reforço para decisões calibradas"). Na Aula 1 vimos RLHF (humanos dando notas) e RLVR (gabarito automático). O RLCD treinaria o modelo para que os números de probabilidade sejam honestos.
 - **"If inteligente":** um `if` (comando de decisão em programação) que, em vez de testar uma regra fixa, consulta o modelo. Ex.: `if (jev diz que é urgente) { mandar para a fila prioritária }`.
 - **Guardrail:** uma "grade de proteção": verificação que bloqueia entradas ou saídas perigosas.
 - **Juiz:** um modelo que avalia a resposta de outro (Aula 3).
-

Slide 8 — LLM x System One (tabela do post)

A tabela foi copiada do post de lançamento. Apresente sem discutir ainda:

- **Velocidade:** LLMs de 3 a 329 segundos; Jev de 70 a 500 **milissegundos** (0,07 a 0,5 segundo).
- **Confiança:** eles chamam os LLMs de "confiantes demais" e o Jev de "calibrado".

Calibrado (vai aparecer muito): quando o modelo diz "tenho 80% de certeza", ele acerta 80% dessas vezes. A previsão do tempo é um bom exemplo: se a meteorologia diz "70% de chance de chuva" em 100 dias, deveria chover em uns 70 deles.

Diga: *"Guardem esses números. A gente volta neles com lupa."*

Slide 9 — Três tipos de pergunta

As três "primitivas" (tipos básicos de pergunta) da API do Jev:

- **Choice** (escolha): "esta notícia é de modelos, ferramentas ou pesquisa?". Devolve a opção escolhida e a chance de cada uma.
- **Score** (nota): "de 0 a 4, quão frustrado está este cliente?". Devolve a nota e a chance de cada nível.
- **Noul** (sim ou não): "este e-mail é urgente?". Devolve a chance de "sim", entre 0 e 1.

Confiança ≠ probabilidade: a probabilidade é a chance de uma opção. A confiança resume o quão "decidido" o modelo está, considerando quantas opções existem.

- 88% numa escolha entre 3 opções → confiança 0,82 (bem decidido).

- 50% / 50% entre 2 opções → confiança 0 (não decidiu nada).
 - Fórmula, se perguntarem: $(\text{número de opções} \times \text{maior probabilidade} - 1) \div (\text{número de opções} - 1)$.
-

Slide 10 — Divisória "2 · Strings x tipos"

String é o nome, em programação, para texto. "Strings x tipos" = texto livre x valores com formato definido.

Slide 11 — Saída estruturada: como funciona por dentro

Hoje quase todo provedor de LLM tem um recurso de **saída estruturada**: você diz qual formato quer e ele garante que a resposta vem nesse formato.

- **Schema**: a "planta" do formato. Ex.: "um objeto com o campo `relevante` (verdadeiro/falso) e o campo `categoria` (uma destas cinco palavras)".
- **Zod**: a biblioteca que usamos em TypeScript para escrever esse schema e validar a resposta.
- **JSON / JSON Schema**: JSON é o formato de dados `{"chave": "valor"}`; JSON Schema é o jeito padrão de descrever esse formato.

Como funciona (ligue com a Aula 1): lembra que o modelo dá uma chance para cada pedaço possível e sorteia um? Com saída estruturada, antes do sorteio, o provedor **apaga** os pedaços que quebrariam o formato. É parecido com o top-p da aula passada, mas o critério é "é válido?" em vez de "é provável?".

A frase-chave: o schema não muda o que o modelo "pensa". Ele só proíbe algumas palavras.

Slide 12 — A armadilha do rótulo

O exemplo mais importante da aula. Rode a demo.

O que aconteceu: pedimos ao modelo uma resposta no formato `{"rotulo": <uma de: modelos, ferramentas, pesquisa, regulacao, mercado, irrelevante>}`.

- Na hora de escrever o valor, o modelo "queria" começar com **"IA"** (72%) ou com **"re"** (de "relevante", 15–20%).
- Nenhuma opção começa com "IA", então esse pedaço foi apagado.
- A única opção que começa com **"re"** é **"regulacao"**. O sistema completou para ela.
- Resultado: notícias de ferramentas e de pesquisa viraram "regulação".

Analogia: é como um formulário com opções fixas em que a pessoa começa a escrever "re..." pensando em "relevante", e o corretor automático completa para "regulação" porque é a única opção que começa assim.

Lição: cuidado com os nomes. Separe perguntas diferentes em campos diferentes ("é relevante?" e, depois, "qual a categoria?"). No projeto, fazemos assim, e o problema some.

Termos na tabela: os valores entre aspas ("IA" , "re") são **tokens** (pedaços de texto, Aula 1). As porcentagens são a chance que o modelo dava para cada um.

Slide 13 — Divisória "3 · Livre x tipado, ao vivo"

Passe direto.

Slide 14 — A mesma notícia, dois jeitos

Rode a demo `demo:livre-vs-tipado`.

- **À esquerda:** a resposta em texto livre. Tem justificativa, é longa (71 a 79 tokens) e demorou 3,6 a 4,6 segundos.
- **À direita:** a resposta tipada. Um objeto pequeno (33 tokens), 2,2 segundos.

Por que o tipado é mais rápido? Aula 1: a saída é gerada um token por vez. Menos tokens = menos tempo e menos custo.

O detalhe importante: os dois **erraram igual** ("ferramentas" em vez de "mercado"). Diga: *"o formato ficou melhor; a inteligência não mudou."*

Slide 15 — Nas 20 notícias do projeto

O resultado do experimento completo (o script `m02:comparar`):

- **Parser:** o pedaço de código que "lê" o texto livre e tenta extrair a decisão. O tipado não precisa de parser.
- **Acertos:** 15 de 20 nos dois jeitos.
- **Latência:** tempo de resposta. O tipado levou cerca de metade.

A história para contar (é verdade e é instrutiva): na primeira versão do parser, eu procurava a categoria no texto inteiro. Como a justificativa citava outras categorias ("...modelos de IA..."), o parser lia a palavra errada. Ele dizia "consegui interpretar 100%" e estava errado em metade. Esse é o perigo real do texto livre: **o erro silencioso**.

Slide 16 — A confiança acompanha o acerto?

Rode a demo `demo:calibracao`.

- O modelo disse, em média, **97%** de confiança. Acertou **75%**. Ou seja, é **confiante demais**.
- Os 5 erros vieram com confiança alta (0,90 a 0,99): a confiança não avisou nada.
- **A tabela da direita:** "se eu só deixar o sistema decidir sozinho quando a confiança for maior que X". Com $X = 0,99$, o sistema decide sozinho 60% das notícias e acerta 92% delas; os outros 40% iriam para um humano. Então a confiança não é inútil: ela **ordena** um pouco os casos, mesmo sem ser um número "honesto".
- **Cobertura:** a fração de casos que o sistema decide sozinho.
- **ECE** (*erro de calibração esperado*): um número que resume o quanto a confiança se afasta do acerto real. 0 é perfeito; 0,22 quer dizer que, em média, a confiança erra por uns 22 pontos percentuais.

Ressalva para dizer: com 20 notícias, os números são instáveis; uma notícia a mais ou a menos muda tudo em 5 pontos. Precisamos de mais dados (Aula 3).

Slide 17 — Três jeitos de pedir confiança a um LLM

- **Logprobs:** as probabilidades internas de cada token (Aula 1). Aqui, no campo verdadeiro/falso, saíram **100% em todos os casos**, certos ou errados. Não ajudou.
- **Autoavaliação:** pedir ao modelo para escrever um número de confiança. Aqui, sempre entre 0,90 e 0,99.
- **Autoconsistência:** perguntar a mesma coisa 5 vezes (com temperatura, para variar) e ver se as respostas concordam. Aqui, ele respondeu igual 5 de 5 vezes **também quando errou**. Isso acontece porque o erro vinha de uma interpretação fixa (ele sempre achava que "caso de empresa" é "ferramentas").

Por que são confiantes demais? Pesquisas mostram que o pós-treino (o RLHF da Aula 1) tende a deixar os modelos mais "seguros de si" do que deveriam. Por isso: **medir antes de confiar**.

Pós-treino: as fases de treino depois do pré-treino (SFT, RLHF, RLVR), que fazem o modelo seguir instruções.

Slide 18 — Divisória "4 · Leitura crítica"

Fornecedor: a empresa que vende o produto. "Números de fornecedor" = números que a própria empresa mediu.

Slide 19 — O que o próprio post ressalva

O post do Jev é honesto ao listar as próprias limitações. Explique cada uma em uma frase:

- **Consulta simplificada / entrada curta:** a demonstração foi montada num cenário que favorece o produto.
- **"No topo do mundo real":** os ganhos que eles mostram são do melhor caso, não do caso típico.
- **Time do próprio produto:** quem montou os testes foi quem fez o produto (viés).
- **Wrapper:** "embrulho", um código intermediário. Os concorrentes foram testados através de um código da TypeSafe que os deixou mais lentos e caros.
- **"0% de erro de tipo" não é medido:** é impossível o Jev devolver uma opção que não existe, porque ele só escolhe entre as opções dadas. Mas escolher **uma opção válida** não é o mesmo que escolher **a opção certa**.

Elogie a transparência: poucos fornecedores listam as próprias ressalvas.

Slide 20 — Onde a própria documentação diz que ele falha

A empresa publica uma página com as limitações da versão atual (jev-1.13):

- **Literal:** responde exatamente o que foi escrito, não o que você quis dizer.
- **Contas, contagens, datas:** a recomendação deles é fazer isso no código, não no modelo.
- **Indireções:** perguntas que exigem vários "pulos" de raciocínio ("o cliente cujo pedido foi feito pelo gerente da loja X...").
- **Ordem das opções:** mudar a ordem das opções pode mudar a resposta.
- **Gerar texto:** não é para isso; use um LLM.

Checklist para qualquer anúncio de IA (diga em voz alta):

1. Quem mediu?
 2. Em que tarefa, com que comparação?
 3. Os autores listam ressalvas?
 4. Dá para reproduzir?
 5. **Funciona nos meus dados?** Esta é a única que decide.
-

Slide 21 — Gerativo ou decisão?

Uma regra simples:

- O resultado é **um texto para uma pessoa ler** (resumo, e-mail)? → LLM gerativo.

- O resultado é **uma escolha que o programa vai usar** (categoria, sim/não, nota)? → decisão tipada.
- As duas coisas? → peça as duas em campos separados (ex.: categoria e justificativa).

Peça à turma um exemplo de cada linha. O "mapa completo" (prompt, skill, agente, decisão) é a Aula 4.

Slide 22 — Divisória "Discussão"

15 minutos de conversa.

Slide 23 — Para conversar (com respostas esperadas)

1. **"A saída é texto ou decisão?"** Não há resposta certa; deixe as pessoas darem exemplos. Bons exemplos de decisão: triagem de tickets, detecção de fraude, aprovação de conteúdo, roteamento de atendimento.
2. **"Se acertou o mesmo tanto, o que ganhamos?"** Formato garantido (sem parser e sem erro silencioso), respostas menores (mais baratas e rápidas) e um lugar para a confiança.
3. **"Disse 0,99 e errou. O que fazer?"** Medir a calibração num conjunto rotulado maior antes de automatizar; definir um limiar a partir dos dados (não do número que o modelo escreve); mandar casos abaixo do limiar para uma pessoa; acompanhar depois de colocar em produção.
4. **"Nunca erra o tipo é verdade por construção, por quê?"** Porque o modelo só consegue devolver opções da lista. Mas pode escolher a opção errada: tipo válido ≠ decisão correta.
5. **"Que evidência pedir antes de trocar?"** Um teste com os **seus** dados rotulados comparando acurácia, calibração, latência e custo dos dois; testar os casos difíceis conhecidos; saber como o fornecedor mediu os números dele.

Se o tempo apertar: faça só a 2, a 3 e a 5.

Slide 24 — Para a próxima aula

- **Exercício:** implementar os cálculos de confiança e de calibração vistos na aula (roda sem modelo, com testes automáticos). O desafio usa a temperatura da Aula 1 para "consertar" um modelo confiante demais (*temperature scaling*: dividir as "notas" do modelo por um número para achatar as probabilidades).
 - **Projeto final, etapa M2:** trocar o texto livre da M1 por uma decisão tipada com confiança, e rodar a comparação nas 20 notícias.
-

Slide 25 — Próxima aula

Gancho para a Aula 3: com 20 exemplos, os números são instáveis; e alguns erros podem ser culpa do rótulo humano (as notícias de "caso de empresa" podem mesmo ser "ferramentas"?). Na próxima aula: como montar um conjunto de testes maior, medir se um prompt melhorou (**evals**) e criar regras que impedem publicar algo errado (**guardrails**).

- **LLM-as-judge**: usar um LLM para dar nota à resposta de outro.
- **Regressão**: quando uma mudança "melhora" uma coisa e piora outra que funcionava.

Glossário rápido

Termo	Em uma frase
Decisão tipada	resposta com formato fixo que o código usa direto
Tipo / string	o formato de um valor / texto
Schema	a "planta" do formato da resposta
Zod	biblioteca TypeScript para escrever e validar schemas
JSON	formato de dados <code>{"chave": "valor"}</code>
Saída estruturada	recurso do provedor que garante o formato da resposta
Decodificação restrita	apagar, antes do sorteio, os tokens que quebrariam o formato
Parser	código que lê um texto e extrai informação dele
System One / Sistema 1	pensamento rápido e intuitivo (Kahneman); nome da classe de modelos da TypeSafe
Jev	o primeiro modelo System One da TypeSafe AI
Early access	produto liberado só para alguns usuários, ainda em teste
RLCD	método de treino do Jev, focado em probabilidades honestas
Choice / Score / Noul	escolha de opção / nota em níveis / sim ou não
Confiança	o quanto o modelo está decidido
Calibrado	quando "80% de certeza" acerta ~80% das vezes
ECE	número que resume o quanto a confiança se afasta do acerto (0 = perfeito)
Brier	outra medida de erro da confiança (0 = perfeito)

Termo	Em uma frase
Cobertura	fração dos casos que o sistema decide sozinho
Limiar	o valor de corte ("só decido sozinho acima de 0,99")
Logprobs	as probabilidades internas dos tokens
Autoavaliação	o modelo escreve a própria confiança
Autoconsistência	perguntar várias vezes e ver se as respostas concordam
Pós-treino	fases de treino depois do pré-treino (SFT, RLHF, RLVR)
Temperature scaling	ajustar a temperatura para a confiança bater com o acerto
Guardrail	verificação que bloqueia entradas ou saídas indesejadas
Fornecedor	a empresa que vende o produto
Wrapper	código intermediário que "embrulha" outro